

Lyndon

Lyndon 分解

定义

首先我们介绍 Lyndon 分解的概念。

Lyndon 串：对于字符串 s ，如果 s 的字典序严格小于 s 的所有后缀的字典序，我们称 s 是简单串，或者 **Lyndon 串**。举一些例子， $a, b, ab, aab, abb, ababb, abcd$ 都是 Lyndon 串。当且仅当 s 的字典序严格小于它的所有非平凡的（非平凡：非空且不同于自身）循环同构串时， s 才是 Lyndon 串。

Lyndon 分解：串 s 的 Lyndon 分解记为 $s = s_1 s_2 \dots s_k$ ，其中所有 s_i 为简单串，并且他们的字典序按照非严格单减排序，即 $s_1 \geq s_2 \geq \dots \geq s_k$ 。可以发现，这样的分解存在且唯一。

Duval 算法

解释

Duval 可以在 $O(n)$ 的时间内求出一个串的 Lyndon 分解。

首先我们介绍另外一个概念：如果一个字符串 s 能够分解为 $s = uv$ 的形式，其中 u 是一个 Lyndon 串，而 v 是 u 的前缀（可能是空串），那么称 s 是近似简单串（pre-simple），或者近似 Lyndon 串。一个 Lyndon 串也是近似 Lyndon 串。

Duval 算法运用了贪心的思想。算法过程中我们把串 s 分成三个部分，其中 u 是一个 Lyndon 串，它的 Lyndon 分解已经记录； v 是一个近似 Lyndon 串； w 是未处理的部分。

过程

整体描述一下，该算法每一次尝试将 w 的首字符添加到 u 的末尾。如果 u 不再是近似 Lyndon 串，那么我们就可以将 u 截出一部分前缀（即 Lyndon 分解）接在 v 末尾。

我们来更详细地解释一下算法的过程。定义一个指针 i 指向 u 的首字符，则 i 从 0 遍历到 (字符串长度) 。在循环的过程中我们定义另一个指针 j 指向 u 的首字符，指针 j 指向 u 中我们当前考虑的字符（意义是 j 在 u 的上一个循环节中对应的字符）。我们的目标是将 w 添加到 u 的末尾，这就需要将 u 与 w 做比较：

1. 如果 $u < w$ ，则将 w 添加到 u 末尾不会影响它的近似简单性。于是我们只需要让指针 i 自增（移向下一位）即可。
2. 如果 $u = w$ ，那么 u 就变成了一个 Lyndon 串，于是我们将指针 i 自增，而让 j 指向 u 的首字符，这样 u 就变成了一个循环次数为 1 的新 Lyndon 串了。
3. 如果 $u > w$ ，则 u 就不是一个近似简单串了，那么我们就把 u 分解出它的一个 Lyndon 子串，这个 Lyndon 子串的长度将是 $j - i$ ，即它的一个循环节。然后把 u 变成分解完以后剩下的部分，继续循环下去（注意，这个情况下我们没有改变指针 i ），直到循环节被截完。对于剩余部分，我们只需要将进度「回退」到剩余部分的开头即可。

实现

下面的代码返回串 s 的 Lyndon 分解方案。



C++Python

```

1 // duval_algorithm
2 vector<string> duval(string const& s) {
3     int n = s.size(), i = 0;
4     vector<string> factorization;
5     while (i < n) {
6         int j = i + 1, k = i;
7         while (j < n && s[k] <= s[j]) {
8             if (s[k] < s[j])
9                 k = i;
10            else
11                k++;
12            j++;
13        }
14        while (i <= k) {
15            factorization.push_back(s.substr(i, j - k));
16            i += j - k;
17        }
18    }
19    return factorization;
20 }

```

```

1 # duval_algorithm
2 def duval(s):
3     n, i = len(s), 0
4     factorization = []
5     while i < n:
6         j, k = i + 1, i
7         while j < n and s[k] <= s[j]:
8             if s[k] < s[j]:
9                 k = i
10            else:
11                k += 1
12            j += 1
13        while i <= k:
14            factorization.append(s[i : i + j - k])
15            i += j - k
16    return factorization

```

复杂度分析

接下来我们证明一下这个算法的复杂度。

外层的循环次数不超过 n ，因为每一次 i 都会增加。第二个内层循环也是 $O(n)$ 的，因为它只记录 Lyndon 分解的方案。接下来我们分析一下内层循环。很容易发现，每一次在外层循环中找到的 Lyndon 串是比我们所比较过的剩余的串要长的，因此剩余的串的长度和要小于 n ，于是我们最多在内层循环 n 次。事实上循环的总次数不超过 n ，时间复杂度为 $O(n^2)$ 。

最小表示法（Finding the smallest cyclic shift）

对于长度为 n 的串，我们可以通过上述算法寻找该串的最小表示法。

我们构建串 s 的 Lyndon 分解，然后寻找这个分解中的一个 Lyndon 串，使得它的起点小于 $n/2$ 且终点大于等于 $n/2$ 。可以很容易地使用 Lyndon 分解的性质证明，子串 $s[i:j]$ 的首字符就是 s 的最小表示法的首字符，即我们沿着 s 的开头往后 $j-i$ 个字符组成的串就是 s 的最小表示法。

于是我们在分解的过程中记录每一次的近似 Lyndon 串的开头即可。



```

1 // smallest_cyclic_string
2 string min_cyclic_string(string s) {
3     s += s;
4     int n = s.size();
5     int i = 0, ans = 0;
6     while (i < n / 2) {
7         ans = i;
8         int j = i + 1, k = i;
9         while (j < n && s[k] <= s[j]) {
10             if (s[k] < s[j])
11                 k = i;
12             else
13                 k++;
14             j++;
15         }
16         while (i <= k) i += j - k;
17     }
18     return s.substr(ans, n / 2);
19 }

```

```

1 # smallest_cyclic_string
2 def min_cyclic_string(s):
3     s += s
4     n = len(s)
5     i, ans = 0, 0
6     while i < n / 2:
7         ans = i
8         j, k = i + 1, i
9         while j < n and s[k] <= s[j]:
10             if s[k] < s[j]:
11                 k = i
12             else:
13                 k += 1
14             j += 1
15         while i <= k:
16             i += j - k
17     return s[ans : ans + n / 2]

```

习题

- [UVa #719 - Glass Beads](#)

本页面主要译自博文 [Декомпозиция Линдона. Алгоритм Дюваля. Нахождение наименьшего циклического сдвига](#) 与其英文翻译版 [Lyndon factorization](#)。其中俄文版版权协议为 Public Domain + Leave a Link；英文版版权协议为 CC-BY-SA 4.0。

本页面最近更新：2024/2/15 22:17:29, [更新历史](#)

发现错误？想一起完善？ [在 GitHub 上编辑此页！](#)

本页面贡献者：[iamtwz](#), [orzAtalod](#), [sshwy](#), [StudyingFather](#), [Chrogeek](#), [diauweb](#), [Enter-tainer](#), [gi-b716](#), [hqztrue](#), [Junyan721113](#), [ksyx](#), [Menci](#), [shawlleyw](#), [Soresen](#), [Suyun514](#), [Xeonacid](#)

本页面的全部内容可在 [CC BY-SA 4.0](#) 和 [SATA](#) 协议之条款下提供，附加条款亦可能应用